卒業論文　2025 年度

Double-Cut-And-Join 操作による ゲノム間距離に関する 線形時間アルゴリズムの再実装

平田 智也
（学籍番号: 62216580）

指導教員　小田 芳彰 准教授

2026 年 2 月 2 日

慶應義塾大学 理工学部 数理科学科 4 年
小田研究室

目次

論文要旨 ・・・

論文要旨

生物学において進化の過程を論じる際，突然変異を経て変化する世代間の影響を考慮することは不可欠である．また，生物学的背景から発生しやすい変異とそうでない変異を区別し，前者を中心に遺伝的な距離を定義し，考察することは自然なアプローチと言える．

Hannenhalli および Pevzner (1995)[3] は，染色体に作用する操作を「転座」「融合」「分裂」「反転」の 4 種類に制限し，遺伝的距離 d_{HP} を定義した．しかしながら，この手法および提案されたアルゴリズムには，計算上の制約や複雑性に関する課題が存在した．

Bergeron ら (2009)[2] は，これらの問題を解決すべく，上記 4 種類の操作を一般化した「Double-Cut-And-Join（以下，DCJ）という単一の操作を導入し，新たなアルゴリズムを提案した．DCJ 操作に基づく距離を d_{DCJ} と呼ぶ．

Bergeron らの手法は，染色体内の特定パターンを検出することで線形時間での操作手順出力を可能にしているが，その実装は高度であり，直感的な理解が困難な側面がある．本論文では，DCJ 理論における Bergeron らと同様の前提条件の下，C 言語を用いたより簡潔な線形時間アルゴリズムの実装について述べる．本稿の前半では d_{DCJ} の算出に特化したものを，後半では操作手順も出力する，より汎用性の高いシンプルなものを紹介する．

実装においては，一部の着想を Bergeron ら (2009) の研究に依拠しつつも，主要なデータ構造および処理ロジックの大部分は独自に構築したものである．

第 1 章　導入

1.1　背景

計算分子生物学において遺伝的距離をモデル化する手法には多くの変遷があるが，その基本的な枠組みは共通している．単純性の原理（Principle of Parsimony）のもとでは，可能な限り小さい回数の変化を知ることが重要になる．つまり，最終的な目標は，二つのゲノム間の最小変異回数を算出し，それらを比較・評価することになる．

本研究に着手するにあたり，Oliveira らによるサーベイ論文 [4] を参照した．この文献を通じて，ゲノムの再編成距離問題の近年の動向に興味を持ち，特に初学者にとって適した入門的なトピックとして紹介されていた DCJ 操作に着目することとした．本論文で実装の対象とした Bergeron らの論文は，この文脈において選定したものである．

1.2　ゲノムおよび染色体の表現方法

比較対象となる二つのゲノムを，それぞれ *source* および *target* と呼ぶ．各ゲノムは染色体（chromosome）の集合であり，各染色体は遺伝子（gene）の並びである．

各遺伝子は正負の符号を持つ整数値で識別する．向きの基準を *target* ゲノムに置き，絶対値の対応づけは任意であるため， *target* が昇べきの順となるようにつける．[1)]

線形染色体と環状染色体を区別するため，本論文では以下の表記を用いる．

- **線形染色体**: $\langle \dots \rangle$ で囲む．
- **環状染色体**: $(\dots)$ で囲む．

注意 1.1　環状染色体は，その要素を巡回させたものと同一である．例えば，(5, 6, 7)，(6, 7, 5)，(7, 5, 6) はすべて同じ環状染色体である．

注意 1.2　重複した遺伝子の存在，向きを考慮するか否か，*source* と *target* で存在する遺伝子が異なる（削除，挿入が発生する）か否か，環状染色体を考えるか，染色体の本数は単一か複数かなど，前提条件で様々なバリエーションがある．今回の前提条件では，複数染色体を認める点と環状染色体も認める点が比較的難しいと言える．

1)　実は，3 章にてこれから紹介する二つのアルゴリズムは，target（B）における遺伝子の番号が昇順に並んでいることが高速化や正しさに本質的に関係していない．そのため，A と B の両方が任意の状態であっても，問題なく適用できる．

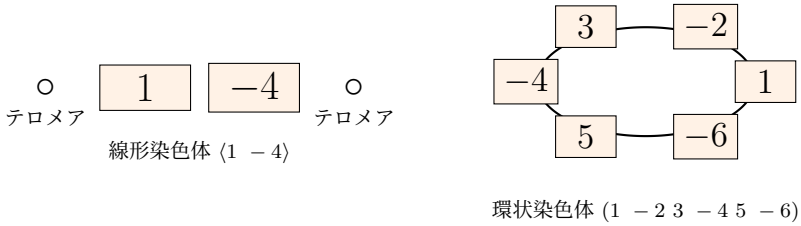


図 1.1　線形染色体と環状染色体の例

1.3　端点（extremity）と隣接（adjacency）

アルゴリズムの実装においては，遺伝子間の「接続関係」を直接扱う方が効率的である．遺伝子 g に対し，二つの **端点（extremity）**を，右側の端点 g および左側の端点 $-g$ と定義する．二つの遺伝子の接続は，これら端点の 2 要素の集合として表現され，これを **隣接（adjacency）** と呼ぶ．すなわち，右側の端点は遺伝子の符号を継承し，左側の端点はその逆符号を持つ．

また，線形染色体の末端については，特殊なシンボル o を用いて**テロメア (Telomere)**[2)]との接続 $\{o, x\}$ として扱う．この定義により，ゲノムは単なる隣接の集合として記述でき，線形・環状の区別や染色体数を意識せずに統合的な操作が可能となる．

なお，実装上の細かな差異について付記すると，**adjacency** は 2 つの **extremity** から構成されるため，いずれのアルゴリズムでも基礎的なデータとして **extremity** が必要となるが，主な操作対象には違いがある．第 3 章のアルゴリズム 1（距離計算）では隣接グラフの構造を調べるため，**adjacency** の走査が行われるが，第 4 章のアルゴリズム 2（操作手順出力）では接続替えの操作を行うため **extremity** を直接操作する．

本研究での実装において，これらの概念は以下の構造体として定義される．

Listing 1.1　extremity 構造体の定義

```
typedef struct extremity
{
    int id; // 遺伝子ID (1 ...)
    int sign; // 向き (1 or -1)
    struct extremity *reality; // ゲノムAにおける接続先
    struct extremity *desire; // ゲノムBにおける接続先
} extremity;
```

Listing 1.2　adjacency 構造体の定義

```
typedef struct adjacency
{
    extremity *left; // 隣接の左端点
```

2)　生物学において染色体の末端を保護する構造体に由来する名称である．

```
    extremity *right; // 隣接の右端点
    struct adjacency *left_to; // グラフ上の左側エッジ
    struct adjacency *right_to; // グラフ上の右側エッジ
    int visited; // 探索用フラグ
} adjacency;
```

1.4 隣接グラフの定義と例

定義 1.3（隣接グラフ $AG(A,B)$） 2 つのゲノム A, B において，それぞれの隣接を頂点集合とし，共通の遺伝子端を持つ頂点間にエッジを引いたグラフを隣接グラフと呼ぶ.

ただし，共通の要素がテロメア o である場合はエッジを引かないものとする．また，2 つの頂点間で 2 つの遺伝子端が共通する場合，それぞれの端点に対応する計 2 本のエッジを引くこととする．したがって，隣接グラフは多重グラフとなり得る.

注意 1.4 それぞれの頂点の次数は高々 2 であるため，隣接グラフはサイクル（Cycle）とパス（Path）のみからなる．また，本論文においてパスやサイクルの「長さ」とは，それを構成する辺の本数を指すものとする.

例 1.5 以下のゲノム A, B を考える （$N = 5$）.

```
A: < 1 -4 > ( 3 -2 ) < 5 >
B: < 1 2 3 > ( 4 5 )
```

これらのゲノムは， 以下のような隣接の集合としても表現できる[3)].

$$A = \{\{o,-1\},\{1,4\},\{-4,o\},\{3,2\},\{-2,-3\},\{o,-5\},\{5,o\}\}$$

$$B = \{\{o,-1\},\{1,-2\},\{2,-3\},\{3,o\},\{4,-5\},\{5,-4\}\}$$

また，これらに基づく隣接グラフの成分は以下のようになる（円形のノードは，青色がゲノム A の隣接，赤色がゲノム B の隣接を表す）.

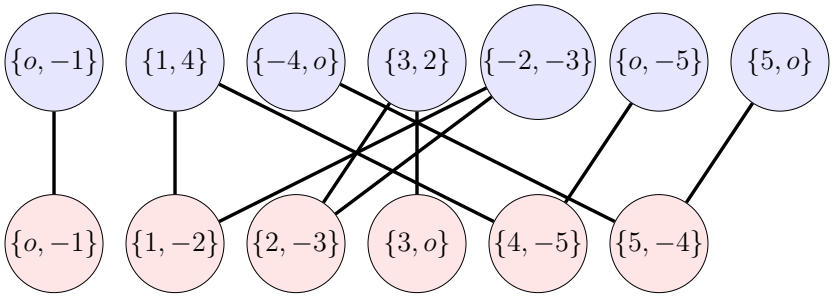


図 1.2 隣接グラフ $AG(A,B)$ の例（上段：ゲノム A，下段：ゲノム B）

3) ゲノムを単なる隣接の集合として表現することで，染色体の種類（線形・環状），本数によらず，統一的に操作することができるようになる.

第 2 章　理論的背景

2.1　d_{DCJ} に関する等式

Bergeron ら (2006)[1] は，DCJ 操作を用いたゲノム距離の計算において，以下の重要な定義および定理を示している．[1)]

定義 2.1（Double-Cut-And-Join）　ゲノムの隣接の集合に対する操作であり，二つの隣接 $\{u, v\}$ と $\{x, y\}$ を切断し，新たに $\{u, x\}$ と $\{v, y\}$，または $\{u, y\}$ と $\{v, x\}$ として再結合する操作を指す．テロメア o を含む隣接に対しても同様に定義される．

また，$\{o, o\}$ との DCJ も認めることで，単なる隣接の切断も DCJ に含めるものとする．

2.2　DCJ 操作の逐次適用例

以下に，例 1.5 における隣接グラフ $AG(A, B)$ に対し，最適な DCJ 操作を順に適用していく様子を図示する．

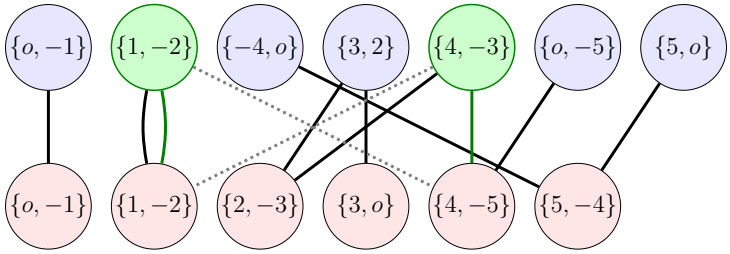


図 2.1　1 回目の DCJ 操作：$\{1, 4\},\ \{-2, -3\} \ \rightarrow\ \{1, -2\},\ \{4, -3\}$

1)　Bergeron らとは異なり，隣接をすべて 2 要素からなる集合と定義したため，DCJ についてもより統一的な定義が可能となった．

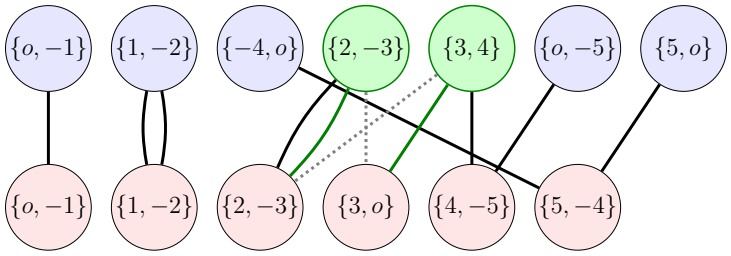


図 2.2　2 回目の DCJ 操作：$\{2,3\},\ \{-3,4\}\ \rightarrow\ \{2,-3\},\ \{3,4\}$

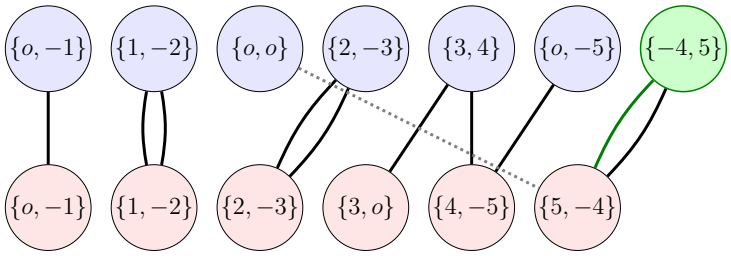


図 2.3　3 回目の DCJ 操作：$\{-4,o\},\ \{5,o\}\ \rightarrow\ \{-4,5\},\ \{o,o\}$

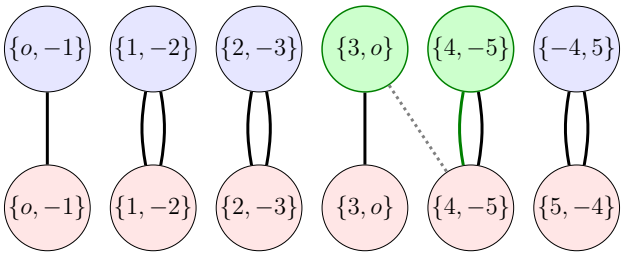


図 2.4　4 回目の DCJ 操作：$\{4,3\},\ \{-5,o\}\ \rightarrow\ \{4,-5\},\ \{3,o\}$

注意 2.2　$d_{\mathrm{DCJ}} = 0$ のとき，テロメア o を含む隣接同士は長さ 1 の奇数長パス，テロメアを含まない隣接同士は長さ 2 のサイクルに揃う．

補題 2.3（成分数の変化）　隣接グラフ $AG(A,B)$ に対する 1 回の DCJ 操作により，連結成分（サイクルまたはパス）の総数は最大で 1 つ増減する．

証明　DCJ 操作は，グラフ上の高々 2 つの頂点に対して行われる局所的なつなぎ替えである．操作対象となる頂点が属する連結成分の種類（パスまたはサイクル）とその数によって，以下の全てのケースに分類できる．それぞれのケースにおける成分数の変化量 Δ を列挙する．[2)]

1. **パスのみに関わる操作**（図 2.5）
   2 つのパス間，あるいは 1 つのパス内での操作．

2)　のちにパスの偶奇が重要になるが，この補題はあくまで連結成分の数のみについての主張であり，偶奇性は問題としていないことに注意されたい．

- (a) 2 つのパスの内部をそれぞれつなぎ替えるケース: 異なるパス間での交換が起きるが，成分数は不変 ($\Delta = 0$).
- (b) 2 つのパスの片方は内部を，もう片方は末端を切断・再結合するケース: 成分数は不変 ($\Delta = 0$).
- (c) 1 つのパスを切断して 2 つのパスにするケース: 成分数の変化は 1 ($\Delta = \pm 1$).

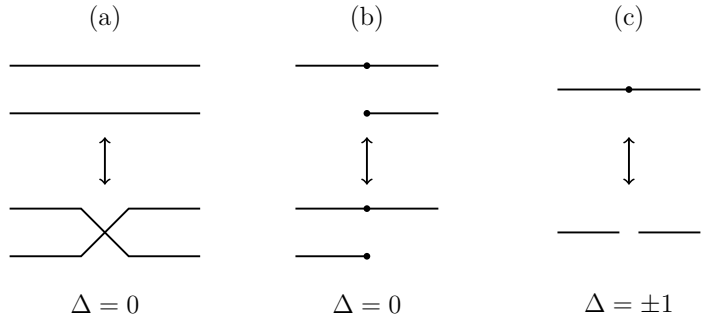


図 2.5　パスのみに関わる DCJ 操作

2. **パスとサイクルが関わる操作**（図 2.6）
   パスとサイクル間の操作.
   - (a) パスから一部を切り出しサイクルとするケース: 成分数の変化は 1 ($\Delta = \pm 1$).
   - (b) パスの両端をつなぎサイクルとするケース: 成分総数はパスが減りサイクルが増える（またはその逆）ため不変 ($\Delta = 0$).
   - (c) パス内の一部の向きを反転させるケース: 成分数は不変 ($\Delta = 0$).

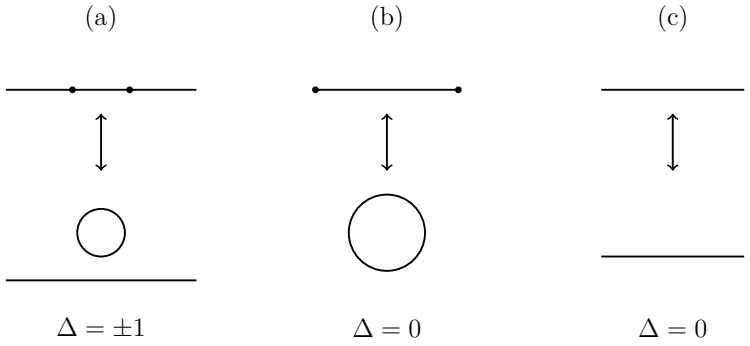


図 2.6　パスとサイクルの変換に関する DCJ 操作

3. **サイクルのみに関わる操作**（図 2.7）

   - (a) 2 つのサイクルを結合するケース: 成分数の変化は 1 ($\Delta = \pm 1$).
   - (b) サイクル内の一部の向きを反転させるケース: 成分数は不変 ($\Delta = 0$).

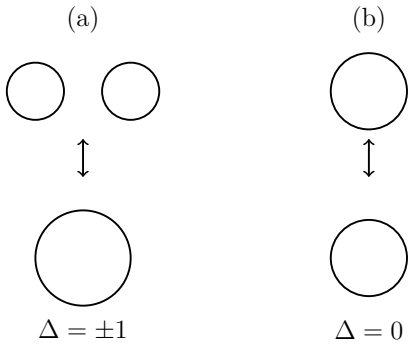


図 2.7　サイクルのみに関わる DCJ 操作

以上の通り，いずれの操作においても成分総数の変化量 $|\Delta|$ は 1 以下である．　□

補題 2.4（奇数長パス数の変化）　隣接グラフにおいて，1 回の DCJ 操作は奇数長パスの数 I を 2 増加，変化なし，あるいは 2 減少させる．

証明　本隣接グラフは，ゲノム A の頂点とゲノム B の頂点の間にエッジを持つ二部グラフである．したがって，パスの長さの偶奇性は両端点が A，B のどちらに属するかによって以下のように定まる．

- **奇数長パス**: 両端点が異なるゲノムに属する．
- **偶数長パス**: 両端点が同じゲノムに属する．

DCJ 操作は単一のゲノム内の隣接に対して作用するため，操作に関与する頂点は同一のゲノムに属する．

1. パス同士の結合・分割（1-a, 1-b, 1-c）　まず，1-c（1 本のパスを 2 本に分割）については，分割後の 2 本のパスの長さが偶数か奇数かで次のように場合分けできる：

- 両方偶数長パスになる場合
- 片方が偶数長，片方が奇数長パスになる場合
- 両方奇数長パスになる場合

それぞれの場合で奇数長パスの数の変化量 ΔI は異なるが，$2, 0, -2$ のいずれかとなる．

1-a, 1-b についても同様に場合分けをすれば容易だが，煩雑になるためここでは省略する．同様に，$2, 0, -2$ のいずれかとなることが確認できる．

2. パスとサイクルの操作（2-a, 2-b, 2-c）　隣接グラフ上の DCJ 操作であることに注意する．

- **パスからのサイクルの切り出し・組込み（2-a）**: パスの内部で自己交差的に接続を組み替えてサイクルを分離する場合，残ったパスの両端点は変更されない．したがってパスの長さの偶奇性も維持され，$\Delta I = 0$.

- **パスの閉路化・サイクルの開路（2-b）**：パスの両端を結合してサイクルにするには，両端が同じゲノムに属する必要がある．これは偶数長パスに対してのみ可能であり，このとき偶数長パスが 1 つ減少しサイクルが増える．奇数長パスの数 I には影響しない（$\Delta I = 0$）．
- **パス内部の反転（2-c）**：パスの両端点は不変であるため，$\Delta I = 0$.

3. サイクルのみの操作（3-a, 3-b）　サイクルのみに関わる操作においてはパスは関与しないため，奇数長パスの数は変化しない（$\Delta I = 0$）．

以上より，いずれのケースにおいても奇数長パスの数 I の変化量は偶数（$-2, 0, 2$）に限られる．　□

補題 2.5（ゲノムの一致条件）　N 個の遺伝子集合からなるゲノム A, B が $A = B$ であるための必要十分条件は，次式が成り立つことである：

$$N = C + \frac{I}{2} \tag{2.1}$$

ここで，C は隣接グラフ $AG(A, B)$ におけるサイクルの数，I は奇数長パス（辺の数が奇数のパス）の数である．

証明　ゲノム A におけるテロメアを含まない隣接の数を a，テロメアを含む隣接の数を t とおくと，$2N = 2a + t$ より，$N = a + t/2$ である．

まず必要性 ($A = B \implies N = C + I/2$) を示す．$A$ と B が同一であれば，すべての隣接は一致する．任意のテロメアを含まない隣接 $\{u, v\} \in A$ は B における同一の隣接と，共有する 2 つの端点を介して 2 重に結合されるため，隣接グラフにおいて長さ 2 のサイクルを形成する．したがって，サイクルの総数は $C = a$ となる．同様に，テロメアを含む隣接 $\{o, u\} \in A$ は B の対応するテロメアと 1 本の辺で結ばれ，長さ 1 のパスとなることから，$I = t$ が従う．ゆえに，$C + I/2 = a + t/2 = N$ が成立する．

次に十分性 ($N = C + I/2 \implies A = B$) を示す．

ゲノム A の隣接数を a，テロメア数を t とする．隣接グラフのサイクル数を C，奇数長パス数を I とする．各隣接はサイクルまたはパスの一部となるため $a \leq C$，各テロメアは奇数長パスの端点となるため $t \leq I$ が成り立つ．また，$N = a + t/2$，仮定より $N = C + I/2$ なので，

$$a + \frac{t}{2} = C + \frac{I}{2}$$

が成り立つ．$a \leq C$，$t \leq I$ かつ両辺が等しいので，$a = C$，$t = I$ が従う．

ここで，隣接グラフの辺の総数は $2N$ 本であり，サイクルは最小長 2，奇数長パスは最小長 1 である．また，サイクルとパスの長さをそれぞれ $l_1, \ldots, l_C$（サイクル），

$m_1, \ldots, m_I$（奇数長パス）とすると，

$$2N = \sum_{i=1}^{C} l_i + \sum_{j=1}^{I} m_j$$

が成り立つ．仮定より $2C + I = 2N$ であるから，

$$\sum_{i=1}^{C} l_i + \sum_{j=1}^{I} m_j = 2C + I = 2N$$

が成り立つ．ここで，すべての i および j について $l_i = 2$，$m_j = 1$ でなければこの等式は成立しない．よって，長さ 3 以上のサイクル，および長さ 2 以上のパスは存在しない．

長さ 2 のサイクルはテロメアを含まない隣接の一致を，長さ 1 のパスはテロメアを含む隣接の一致をそれぞれ意味する．以上より，すべての隣接が完全に一致するため，$A = B$ が示された． □

補題 2.6（DCJ 距離の下限） 任意の 2 つのゲノム A, B について，DCJ 距離は以下を満たす：

$$d_{\mathrm{DCJ}}(A, B) \geq N - \left(C + \frac{I}{2} \right) \tag{2.2}$$

証明 補題 2.3 および補題 2.4 より，1 回の DCJ 操作でサイクル数 C と奇数長パス数 I が同時に変化することはない．したがって，量 $C + I/2$ の 1 操作あたりの増加量は最大でも 1 である（C が 1 増えるか，I が 2 増える場合）．目標状態では補題 2.5 より $N = C + I/2$ であるため，初期状態からの差分を埋めるために少なくとも右辺の回数の操作が必要となる． □

定理 2.7（DCJ 距離公式） 2 つのゲノム A, B 間の最短の DCJ 操作回数（DCJ 距離）$d_{\mathrm{DCJ}}(A, B)$ は，次式で与えられる：

$$d_{\mathrm{DCJ}}(A, B) = N - \left(C + \frac{I}{2} \right) \tag{2.3}$$

この距離を達成する最短の並べ替え手順は，線形時間 $O(N)$ で計算可能である．

証明 補題 2.6 により下限が示されているため，この回数でソートを行う手順が存在することを示せばよい．まず，$A \neq B$ である限り，B に存在して A に存在しない隣接が必ず存在する（$\because A$ と B が一致せず，かつ全ての隣接が一致している状況は存在しない）．第 4 章で詳述するアルゴリズムは，そのような不一致箇所を解消するように操作を行うことで，常に $C + I/2$ を 1 ずつ増加させる操作（C を 1 つ増加させる，あるいは I を 2 つ増加させる操作）を選択し続けることができる．したがって，下限値でのソートが可能であり，これが最短距離となる． □

2.3 距離公式に関する考察

(2.3) 式は，d_{DCJ} を計算する上で，隣接グラフの初期状態さえ構築できれば，実際にグラフを物理的に変形させる手順を踏まずとも，グラフ内の連結成分（サイクルおよびパス）を走査し C, I を計数するだけで距離が確定することを示している．

この知見に基づき，まずアルゴリズム 1 を提案する．

第 3 章　アルゴリズム 1：距離計算の実装

本アルゴリズムは，隣接グラフを構築し，そこに含まれるサイクルおよび奇数長パスを線形時間で探索するものである．

3.1　入力形式

本実装では，以下のフォーマットに従うテキストファイルを入力とする．1 行目に遺伝子数 N，2 行目にゲノム A，3 行目にゲノム B を記述する．以下はその例である：

```
5
< 1 -4 >( 3 -2 )< 5 >
< 1 2 3 >( 4 5 )
```

ここで `< ... >` は線形染色体，`( ... )` は環状染色体を表し，内部の数字は遺伝子 ID およびその符号（向き）を示す．[1)]

3.2　データ構造

実装においては，以下の 3 つの要素を中核とする．

- **extremity**: 遺伝子の端点を表す構造体．ID と符号を保持する．
- **adjacency**: 2 つの extremity のペア．探索を効率化するため，各隣接が $source$ と $target$ のどちらに属するか，および訪問済みかどうかのフラグを持つ．
- **Map**: 特定の端点がどの adjacency に含まれているかを定数時間で検索するための整数値配列．[2)]

これらのデータ構造は，以下の手順によって入力から構築される．

---

1)　あくまで私の実装の例であり，適切な処理がなされる限り入力形式には特に制約はない．

2)　実行時間を $O(N)$ に収めるための工夫である．

Algorithm 1: EXT 配列（端点配列）の構築

入力: 入力ファイルの 1 行目・2 行目

出力: EXT: サイズ $2N$ の extremity 配列

$N \leftarrow$ 1 行目から読み込む;

$EXT \leftarrow$ サイズ $2N$ の配列を初期化;

2 行目をパースし，A の遺伝子 ID・符号ごとに，**foreach 遺伝子** g **do**

    $EXT[2(g-1)] \leftarrow g$ の左端点, $EXT[2(g-1)+1] \leftarrow$ 右端点;

return EXT;

Algorithm 2: ADJ 配列（隣接配列）と Map 配列の構築・接続

入力: EXT 配列, 入力ファイルの 2 行目・3 行目

出力: ADJ: adjacency 配列, Map: 端点→ ADJ インデックスの配列

$ADJ \leftarrow$ 必要なサイズの配列を初期化;

$Map \leftarrow$ サイズ $2N$ の配列を初期化;

// ゲノム `A` の隣接を `ADJ` 配列に格納

2 行目をパースし，A の隣接ごとに，**foreach 隣接** $\{u, v\}$ **do**

    $ADJ[i] \leftarrow u, v$ を持つ新規 adjacency ノード;

    $Map[u] \leftarrow i$, $Map[v] \leftarrow i$;

    $i \leftarrow i + 1$;

// ゲノム `B` の隣接を `ADJ` 配列のノード間に接続

3 行目をパースし，B の隣接ごとに，**foreach 隣接** $\{x, y\}$ **do**

    $ADJ[Map[x]]$ と $ADJ[Map[y]]$ の間にエッジ（$left_to$ / $right_to$）を張る;

return ADJ, Map;

Algorithm 3: 隣接グラフを用いた DCJ 距離の計算

入力: ゲノム A, ゲノム B

出力: DCJ 距離 d_{DCJ}

```
N ← 遺伝子数;
AG(A, B) ← A と B の隣接集合から隣接グラフを構築;
C ← 0, I ← 0;
foreach 未訪問の隣接ノード v ∈ AG(A, B) do
    L ← 0;
    cur ← v;
    while cur が未訪問 do
        cur を訪問済みにする;
        L ← L + 1;
        next ← cur に隣接する未訪問ノード;
        if next が存在しない then
            break;
        cur ← next;
    if 探索が開始点に戻った（サイクルを発見） then
        C ← C + 1;
    else if 端（テロメア）に到達した（パスを発見） then
        if L が奇数 then
            I ← I + 1;
d_DCJ = N − (C + I/2);
return d_DCJ;
```

3.3 正当性と時間計算量

正当性については，定理 2.7 の結果による．

本アルゴリズムの探索部分は，各エッジをちょうど一度ずつ走査するため，エッジの数がちょうど 2N であることから，時間計算量は $O(N)$ となる．

第 4 章　アルゴリズム 2：操作手順の出力

[1] の理論を基に，実際の DCJ 操作手順を逐次的に出力するアルゴリズムを実装した.

4.1　実装方針

本アルゴリズムでは，現在のゲノム状態を目標のゲノム状態へ貪欲法[1)] により近づけていく．具体的には，特定の接続が目標とする接続となっていない場合，DCJ 操作を適用してその接続を確立させていく.

$exts$ および $goals$ は，前述の Algorithm1 で構築した EXT 配列を用いる.

1)　Bergeron ら [1] の言葉を引用した.

Algorithm 4: DCJ 操作によるゲノムソート手順の生成

入力: 現在のゲノム A の端点集合 $exts$, 目標状態 $goals$

出力: DCJ 距離 d_{DCJ}

$n \leftarrow$ 遺伝子数, $c \leftarrow 0$;

// 操作ログは各 DCJ 操作のたびに出力する

// 第 1 段階：目標が通常の端点である接続を優先処理

for $i = 0$ **to** $2n - 1$ **do**

  $src \leftarrow exts[i]$;

  $tar \leftarrow goals[i]$;

  if tar **が有効な端点 *and*** src **の現在の接続相手** $\neq tar$ **then**

    DoubleCutJoin(A, src, tar);

    操作内容をログとして出力;

    $c \leftarrow c + 1$;

// 第 2 段階：目標がテロメアである接続を処理

for $i = 0$ **to** $2n - 1$ **do**

  $src \leftarrow exts[i]$;

  $tar \leftarrow goals[i]$;

  if tar **がテロメア *and*** src **の現在の接続相手** $\neq tar$ **then**

    DoubleCutJoin(A, src, tar);

    操作内容をログとして出力;

    $c \leftarrow c + 1$;

$d_{\mathrm{DCJ}} \leftarrow c$;

return d_{DCJ};

4.2 正当性と時間計算量

正当性については，定理 2.7 の証明を参照されたい．

また，時間計算量について述べると，本アルゴリズムが行う操作回数は $d_{\mathrm{DCJ}}(A, B)$ に一致する．補題 2.6 より，これは最大でも N 回である．各ステップにおける端点の参照や DCJ 操作は定数時間で実行でき，アルゴリズム全体での繰り返しの回数も遺伝子数 N に比例するため，全体の時間計算量は $O(N)$ となる．

第 5 章　評価

本研究で実装したプログラムの性能を検証するため，異なるスケールのデータセットを用いて実行時間およびメモリ使用量の計測[1)] を行った．

5.1　実験環境

実験に使用した計算機のスペックは以下の通りである．[2)]

- **機種名**: MacBook Air (15-inch, M3)
- **チップ**: Apple M3（8 コア: パフォーマンス 4，効率性 4）
- **メモリ**: 16 GB
- **OS**: macOS (Darwin Kernel Version 24.6.0)

5.2　データセット

遺伝子数 N を 10 から $100,000$ まで変化させ，ランダムに生成したゲノム対を入力として使用した．

5.3　実験結果

データの読み込みおよび構造体の構築を含めた全体の実行時間（表 5.1）と，構築済みのデータ構造に対するアルゴリズム部分のみの実行時間（表 5.2）をそれぞれ計測した．なお，メモリ使用量はピーク時の値を Byte から MB に換算して記載した．また、実行時間については、3 桁で四捨五入している。

1)　メモリ使用量の計測はオリジナルな処理を用いており，ヒープ領域に割り当てられたものだけを計上している．それ以外のスタック領域等に割り当てられたものについては，誤差の範囲のサイズであるため無視して構わない．

2)　私物である．実行環境の違いから，Windows 環境の方が倍程度遅い傾向にあったが，整数のマシン語長の違い等により，メモリ使用量は Windows 環境の方が小さかった．

表 5.1　全体実行時間（読み込み・構築含む）とメモリ使用量

遺伝子数 (N)	実行時間 (sec)	メモリ使用量 (MB)
10	0.000786	0.003
100	0.001302	0.03
1,000	0.007157	0.26
10,000	0.179437	2.55
100,000	20.827368	25.55

表 5.2　アルゴリズム実行時間（読み込み・構築除く）

遺伝子数 (N)	実行時間 (sec)
10	0.000681
100	0.000911
1,000	0.002232
10,000	0.004517
100,000	0.051467

5.4　考察

実験結果より，メモリ使用量に関しては遺伝子数 N に対して線形増加の傾向を示しており，$N = 100,000$ の規模でも約 26 MB 程度と，一般的な PC 環境において十分に動作する範囲に収まっていることが確認できる．

実行時間について，データの読み込みを含めた場合（表 5.1）は $N = 100,000$ で約 20 秒を要しているのに対し，アルゴリズム部分のみ（表 5.2）では約 0.05 秒で処理が完了している．この乖離の主な要因は，入力処理におけるテキストデータのパース（構文解析）にあると考えられる．本実装では入力の容易さと可読性を重視し，10 進数のテキスト形式を採用している．しかし，遺伝子数 N が増大すると，個々の遺伝子 ID を表す文字数（桁数）も $\log_{10} N$ のオーダーで増加するため，テキストデータの総容量およびパースにかかる時間は $O(N \log N)$ で増加する．

厳密な時間計算量理論の観点に立てば，$N \to \infty$ において数値を表現するビット長自体が増大するため，いかなる形式であれ完全な線形時間の達成は困難である．しかし，アルゴリズムの理論解析で一般に用いられる計算モデル（Word RAM [3] Model）では，マシン語長に収まる整数の操作は定数時間 $O(1)$ とみなされる．この前提に近づけ，実用的な速度を得るためには，本来であればバイナリ形式の採用などによる入出力の最適化が為されるべきである．本研究では，データ作成の簡便さとデバッグ時の可読性

3）Random Access Machine の略.

を優先し，テキスト入力からのパースを用いた結果，この入出力オーバーヘッドが残存することとなった.[4)]

一方，本手法の核心となる DCJ 距離計算および操作手順生成アルゴリズムそのものは，概ね線型時間（O(N)）で動作していると結論付けられる．表 5.2 において，N が小さい範囲ではターミナルへの標準出力や関数呼び出し等のオーバーヘッドが支配的であるからか，厳密な線型性は観測されない．しかし，計算負荷が大きくなる $N = 10,000$ から $N = 100,000$ の推移に注目すると，実行時間は約 11.4 倍と，データ量の増加に概ね比例している．この結果は，アルゴリズムの時間計算量が理論通り線型時間であることを示唆する．

4） $N = 100000$ では想像以上に大きな時間がかかった．そして $N = 1000000$ では私の忍耐力が持続する時間内には終了しなかったため，メモリ量など，別の部分でボトルネックが発生していたと思われる．いずれにしろ，大きな N でも問題なく実行できるプログラムを実装するには，入力形式を大きく工夫する必要があることは明白である．

参考文献

[1] A. Bergeron, J. Mixtacki, and J. Stoye, “A unifying view of genome rearrangements,” in: *Proceedings of WABI 2006*, LNBI, vol. 4175, Springer Verlag, 2006, pp. 163–173.

[2] A. Bergeron, J. Mixtacki, and J. Stoye, “A new linear time algorithm to compute the genomic distance via the double cut and join distance,” *Theoretical Computer Science*, 410 (2009), pp. 5300–5316.

[3] S. Hannenhalli and P. A. Pevzner, “Transforming men into mice (polynomial algorithm for genomic distance problem),” in: *Proceedings of FOCS 1995*, IEEE Press, 1995, pp. 581–592.

[4] A. R. Oliveira, K. L. Brito, A. O. Alexandrino, G. Siqueira, U. Dias, and Z. Dias, “Rearrangement Distance Problems: An Updated Survey,” *CSUR*, 2024.

付録 A　ソースコードの公開

本研究で実装した線形時間アルゴリズムのソースコード，および検証に使用したデータセットは，以下の GitHub リポジトリにて公開している．本実装は C 言語により記述されており，ビルドには `make` コマンドを利用するが，通常のコンパイルも可能である．`README.md` を参照いただきたい．

- **リポジトリ名**: `dcj_bergeron`
- **URL**: `https://github.com/hiratannnnnn/dcj_bergeron`
- **最終編集日**: 2026 年 1 月 28 日

なお，本リポジトリには以下のファイルが含まれる．

1. `lib`: 汎用関数
2. `srcs/solve.c`: アルゴリズム 1（距離計算）およびアルゴリズム 2（操作手順出力）の主要実装
3. `srcs/double_cut_join.c`: DCJ 操作の基本機能の実装
4. `srcs/contexts.c`: 実行コンテキストおよびデータ構造の初期処理
5. `srcs/utils_*.c`: 入出力処理やデータ構造操作の補助関数群
6. `include/`: 構造体（extremity, adjacency 等）及び関数のプロトタイプ宣言を記載したヘッダファイル群
7. `benchmark_author`: 私の実行環境でのベンチマーク
8. `example.txt`: 例 1.5 のテキスト形式
9. `LICENSE`: 簡単な謝辞
10. `main_make_ex.c`: テスト用ファイルランダム生成用プログラム
11. `main.c`: プログラムのエントリーポイント
12. `Makefile`: ビルド用スクリプト
13. `memo.md`: 実装メモ
14. `README.md`: 実行方法および環境要件の解説